

Mining Intrinsic Rewards from LLM Hidden States for Efficient Best-of-N Sampling

Jizhou Guo*

Zhiyuan College, Shanghai Jiao Tong University
Shanghai, China
sjtu18640985163@sjtu.edu.cn

Hanchen Yang

Tongji University
Shanghai, China
neoyang@tongji.edu.cn

Zhaomin Wu[†]

National University of Singapore
Singapore
zhaomin@nus.edu.sg

Philip S. Yu

University of Illinois Chicago
Chicago, IL, USA
psyu@uic.edu

Abstract

Best-of-N sampling is a powerful method for improving Large Language Model (LLM) performance, but it is often limited by its dependence on massive, text-based reward models. These models are not only computationally expensive but also data-hungry, requiring extensive labeled datasets for training. This creates a significant data challenge, as they overlook a rich, readily available data source: the LLM’s own internal hidden states. To address this data and efficiency gap, we introduce SWIFT (Simple Weighted Intrinsic Feedback Technique), a novel and lightweight method that learns a reward function directly from the rich information embedded in LLM hidden states. Operating at the token embedding level, SWIFT employs simple linear layers to effectively distinguish between preferred and dispreferred generations, eliminating the need for computationally intensive text-based modeling. Extensive experiments on standard benchmarks show that SWIFT outperforms existing baselines (12.7% higher accuracy than EurorRM-7B on MATH dataset) while using less than 0.005% of their parameters. Its robust scalability, compatibility with certain closed-source models via logit access, and ability to combine with traditional reward models for additional performance highlight SWIFT’s practical value and contribution to more efficient data-driven LLM post-training. Our code is available at <https://github.com/aster2024/SWIFT>.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Natural language processing**; *Learning from implicit feedback*; *Learning latent representations*.

Keywords

Large Language Models, Knowledge Discovery, Reward Modeling

ACM Reference Format:

Jizhou Guo, Zhaomin Wu, Hanchen Yang, and Philip S. Yu. 2026. Mining Intrinsic Rewards from LLM Hidden States for Efficient Best-of-N Sampling. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770854.3780302>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18140886>.

1 Introduction

The proliferation of Large Language Models (LLMs) has unlocked unprecedented capabilities in various domains, from complex reasoning to user assistance [26, 41, 56]. To further refine and enhance these capabilities after initial pretraining, various post-training techniques are employed, including methods that scale computation at test-time to improve performance on difficult tasks [25]. A key one of these test-time scaling strategies is Best-of-N sampling, in which a reward model selects the best response from a set of candidates [43]. However, the effectiveness of this approach is significantly constrained by the reward models themselves. Current methods rely on large, independently trained neural networks that predict rewards based on generated text. These models are not only computationally expensive but also data-hungry, requiring massive datasets and incurring substantial overhead during both training and inference [14, 33, 57]. This exclusive reliance on textual data necessitates complex architectures to capture subtle errors, posing a major barrier to efficient LLM enhancement.

We argue that this paradigm overlooks a rich, readily available data source: the task-performing LLM’s own internal hidden states. During generation, an LLM naturally emits internal signals that reflect its confidence of its answer and the quality of its reasoning. Seminal work has shown that this “internal knowledge” is not only information-rich but is often captured in a linearly separable representations within the model’s hidden states [48, 55]. Our own preliminary experiments support this finding (Section 4.1), revealing a clear linear relationship between hidden states and reasoning correctness. This property is significant, as it points to a path for highly efficient reward modeling: rather than collecting ever-larger text datasets to train ever-larger external models, we can mine the intrinsic signals already present within the LLM itself.

*Work done during an internship at University of Illinois Chicago.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '26, Jeju Island, Republic of Korea*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2258-5/2026/08

<https://doi.org/10.1145/3770854.3780302>

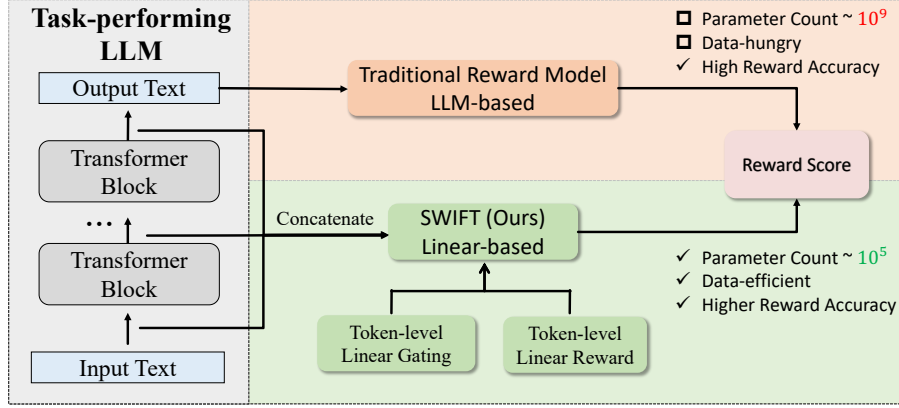


Figure 1: An illustration comparing traditional reward model and SWIFT.

However, leveraging this insight introduces a fundamental architectural challenge. Traditional reward models are designed to process discrete text tokens, creating a mismatch with the continuous, high-dimensional vectors of hidden states. Addressing this mismatch requires a new architecture tailored specifically to extract meaningful signals from these hidden representations. While prior work has explored hidden states to improve reasoning, such methods typically focus on single, final-answer tokens or rely on subjective definitions of reasoning steps [2, 5]. As a result, they offer limited performance gains. They also do not produce a direct reward signal suitable for Best-of-N sampling.

To address this challenge, we propose the Simple Weighted Intrinsic Feedback Technique (SWIFT), a novel architecture specifically designed to learn from hidden states at the token level (Figure 1). For each token, SWIFT learns both a linear gating value and a linear reward projection. The final reward is computed as a weighted average of these token-level rewards, offering a holistic, fine-grained assessment of the entire generation. Our experiments show that **SWIFT outperforms baseline reward models** (demonstrated 12.7% higher accuracy in average on MATH dataset than EurorRM-7B, a strong open-source reward model on math and code domain [53]) while using **fewer than 0.005% of their parameters**. It is exceptionally data-efficient, can be trained with only a handful of examples, and achieves **orders-of-magnitude improvements in computational efficiency** (FLOPs) compared to existing baselines.

Due to the strict page limits of the conference proceedings, we omit extended appendix material in this camera-ready version; the full version is available at <https://arxiv.org/abs/2505.12225> [15]. Our code is available at <https://github.com/aster2024/SWIFT>. In summary, our main contributions are:

- We introduce SWIFT, a token-level reward model that directly mines LLM hidden states. We demonstrate SWIFT’s orthogonal design can augment traditional reward models to achieve further performance gains.
- We extend SWIFT to operate with logit-only access, demonstrating its practical applicability to a broader range of models, including certain closed-source LLMs.
- Our experiments demonstrate that SWIFT is highly data-efficient, achieving strong performance with minimal training data. Furthermore, it exhibits robust scalability with

performance improvement from both training-time and test-time scaling.

2 Related Work

Reward Modeling for Language Models’ Reasoning. The rapid advancement of Large Language Models (LLMs) in complex tasks, such as answering user’s questions [32, 42] and reasoning [50, 58], has highlighted the pivotal role of reward modeling in both training and post-training inference [8, 21]. Current reward models typically build upon an LLM backbone, with substantial parameter count and requires significant amount of data for training, which introduces high computational costs [14, 33]. Although prior work has proposed various approaches to mitigate such issue [6, 12], achieving a satisfactory balance between efficiency and performance remains a challenge. This work introduces a novel perspective by leveraging internal representations to achieve a significant improvement in efficiency with satisfactory performance.

Understanding and Leveraging the Internal Representations of Language Models. The rapid development of LLMs has spurred considerable research into their internal representations [10, 22]. Hidden states of LLMs have been effectively utilized in diverse fields, including safety alignment [4, 60], faithfulness and hallucination detection [5], knowledge editing [29, 52], and knowledge distillation or transfer [9, 47]. By contrast, we effectively leverage the hidden states in intermediate layers of LLMs as a direct signal for reward modeling in reasoning tasks.

3 Problem Statement

A common approach to enhance LLM reasoning is best-of-N sampling [43]. Let $\mathcal{V}$ be the token vocabulary and let $\mathcal{D}$ be a distribution over problem-answer pairs $(x, y) \in \mathcal{V}^n \times \mathcal{Y}$, where y is the hidden reference solution for question x . Conditioned on x , the base LLM π draws N reasoning paths independently (i.i.d. draws from $\pi(\cdot | x)$), producing $r_i = (r_{i,1}, \dots, r_{i,m_i}) \in \mathcal{V}^{m_i}$, $i = 1, \dots, N$, where m_i is the random length of the i -th path. We write $\mathcal{V}^* = \bigcup_{\ell=0}^{\infty} \mathcal{V}^{\ell}$. Simultaneously, π emits hidden states $h_i = (h_{i,1}, \dots, h_{i,m_i}) \in (\mathbb{R}^{L \times d})^{m_i}$, where L is the number of transformer layers, d is the per-layer hidden-state dimension, and each $h_{i,j} \in \mathbb{R}^{L \times d}$ is the stacked hidden state after emitting $r_{i,j}$. Define the correctness indicator $F : \mathcal{V}^* \times$

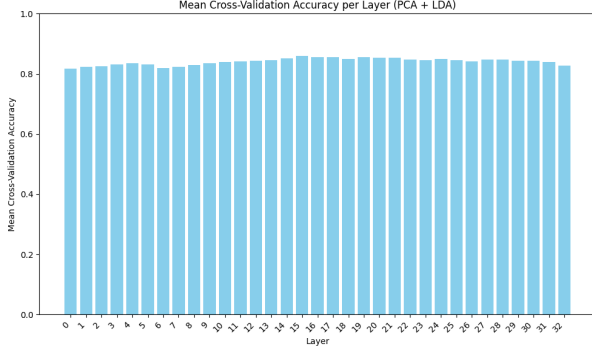


Figure 2: Cross-validation accuracy of the PCA+LDA pipeline for predicting correctness at each layer of Llama-3.1-8B. The results demonstrate that hidden states contain information about reasoning correctness with linear representation.

$\mathcal{Y} \rightarrow \{0, 1\}$, s.t. $F(r, y) = 1 \iff r$ yields reference answer y . We write $(\mathbb{R}^{L \times d})^* = \bigcup_{k=0}^{\infty} (\mathbb{R}^{L \times d})^k$. A parametric reward model $R_{\theta} : (\mathbb{R}^{L \times d})^* \rightarrow \mathbb{R}$ assigns a scalar score to each h_i (hence for each r_i). Note that neither π nor R_{θ} sees y . At test time we select $i^* = \arg \max_{1 \leq i \leq N} R_{\theta}(h_i)$ and return r_{i^*} . Our goal is to choose θ to maximize the probability that r_{i^*} is correct (expectation of $F(r_{i^*}, y)$) under D :

$$\max_{\theta} \mathbb{E}_{(x, y) \sim D} \mathbb{E}_{(r_1, \dots, r_N) \sim \pi(\cdot | x)} [F(r_{\arg \max_i R_{\theta}(h_i)}, y)]. \quad (1)$$

4 Methods

4.1 Motivation

We first present a preliminary experiment demonstrating the potential for enhancing reasoning using hidden states. Recent work has shown that LLM hidden states contain information about the correctness probability of a given reasoning path, often exhibiting *linear* representations [48, 55]. To verify this, we designed a simple pipeline to predict the correctness of reasoning steps based on the LLM’s hidden states.

Our pipeline extracts hidden states h_t^{ℓ} for each token t at each layer ℓ from the Llama-3.1-8B-Instruct model [13] during reasoning on the MATH dataset [17], averages the hidden states across all tokens within each reasoning path to obtain a single vector representation $\bar{h}^{\ell}$ for each layer ℓ , then reduces $\bar{h}^{\ell}$ for each ℓ to 50 dimensions using Principal Component Analysis (PCA) [46], and classifies the reasoning as correct or incorrect using Linear Discriminant Analysis (LDA) [3]. The pipeline is linear to verify that the hidden states can encode a *linear* representation for confidence of the answer.

We performed 5-fold cross-validation on 3000 instances of the MATH dataset. For each layer ℓ , we trained the pipeline on the training folds and evaluated the classification accuracy on the held-out test fold. Figure 2 presents the average cross-validation accuracy for each layer.

As shown in Figure 2, the accuracy is approximately 80% on each layer, indicating a strong signal for reasoning correctness within the

hidden states. This result verifies the aforementioned theory and motivates the development of a novel, lightweight, *linear* reward model that directly leverages this information, as described in the following sections.

4.2 Our Approach - SWIFT

Therefore, building upon the observation that LLM internal representations exhibit linear properties, we designed SWIFT (Simple Weighted Intrinsic Feedback Technique) model to be a parameter-efficient, linear model, which learns a token-level reward function directly from the concatenated hidden states.

For each token t in a reasoning path, we concatenate and flatten the hidden states (after residual connection) from all L layers: $h_t = [h_t^1; h_t^2; \dots; h_t^L] \in \mathbb{R}^{Ld}$, where d is the dimension of the hidden state. We then apply a linear transformation to obtain a gating value g_t and a token-level reward r_t :

$$\begin{pmatrix} \tilde{g}_t \\ r_t \end{pmatrix} = W_{\text{SWIFT}} h_t + b_{\text{SWIFT}} \quad (2)$$

where $W_{\text{SWIFT}} \in \mathbb{R}^{2 \times Ld}$ is the weight matrix, $b_{\text{SWIFT}} \in \mathbb{R}^2$ is the bias vector, $\tilde{g}_t$ is the pre-activation gating value, and r_t is the token-level reward. The gating value is then passed through a sigmoid function [16]:

$$g_t = \sigma(\tilde{g}_t) = \frac{1}{1 + e^{-\tilde{g}_t}} \quad (3)$$

The final reward R for the entire reasoning path is the weighted average of the token-level rewards, using the gating values as weights:

$$R = \frac{\sum_{t=1}^T (g_t \cdot r_t)}{\max \left(\sum_{t=1}^T g_t, \epsilon \right)} \quad (4)$$

where T is the number of tokens, and ϵ is a small constant (e.g., 10^{-8}) for numerical stability. This model only contains $O(L \times d)$ parameters, which is significantly fewer than those in traditional reward models.

Since different tokens contribute differently to the overall correctness, the gating values allow the model to assign higher weights to more important tokens. We provide additional ablations on the gating mechanism in the full version.

We use the binary cross-entropy with logits loss [38] for training. Let $y \in \{0, 1\}$ denote the correctness label of a reasoning path, where $y = 1$ indicates a correct path and $y = 0$ indicates an incorrect path. For a path with hidden state representation h , the model predicts a scalar logit score $R(h) \in \mathbb{R}$. The binary cross-entropy with logits loss is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\sigma(R(h_i))) + (1 - y_i) \cdot \log(1 - \sigma(R(h_i)))] \quad (5)$$

where $\sigma(x) = \frac{1}{1 + e^{-x}}$ is the sigmoid function.

Although binary cross-entropy with logits loss generally yields good results, we also observe that alternatives such as DPO [39] and hinge loss [40] can perform competitively in some settings. Given that cross-entropy performs best on average in our experiments,

we adopt it as our primary loss function in the main text; extended loss comparisons are provided in the full version.

5 Experiments

5.1 Experimental Setup

Our experiments are conducted across mathematical reasoning, code comprehension, commonsense abilities and symbolic reasoning tasks. We evaluate our method on the MATH [17], GSM8K [7], AQuA_RAT [23], Imbue Code Comprehension [18], HellaSwag [54] and CoinFlip [45] datasets. The models used for generation are Llama-3.2-3B-Instruct [30], Llama-3.1-8B-Instruct [13], and Ministral-8B-Instruct [31]. The generation temperature is 1.0 with `top_p` to be 0.9. We use 6000 instances (each with 8 reasoning paths) for training and 500 instances for evaluation. When training SWIFT, we use early stopping [51] based on validation set performance to mitigate overfitting. Additional implementation details are available in the full version.

We compare our method against six recent open-source reward models: EururRM-7B [53], Skywork-Reward-Llama-3.1-8B-v0.2 [24], Starling-7B [59], UltraRM-13B [8], RLHFlow-8B-Deepseek [49], Math-Shepherd-7B [44]. These baselines are very parameter heavy and trained on extensive datasets, while achieving leading performance among open-source reward models. **These six baseline reward models have all been explicitly trained on data related to reasoning and code domains. Their enhanced performance in these specific areas is well-documented and validated in their respective original publications.** It is particularly noteworthy that the development of EururRM-7B, Ultra-13B, RLHFlow-8B-Deepseek, and Math-Shepherd-7B involved training and evaluation on the MATH and GSM8K datasets. This corresponds directly with the datasets used in our work, confirming the relevance of these models as strong baselines.

For parameter count and training data size of baselines compared against SWIFT, see Table 3 for details. We also include a fine-tuned EururRM-7B baseline in Table 1–2.

5.2 Accuracy

Table 1 reports the Best-of-N performance on mathematics domain of each reward model on the MATH, GSM8K, and AQuA_RAT datasets, and Table 2 reports their performance on other general domains, including code comprehension (Imbue Code Comprehension dataset), commonsense (HellaSwag dataset) and symbolic reasoning (CoinFlip dataset). Our SWIFT model **outperforms all baseline reward models** across most BoN settings. For example, SWIFT is 12.7% more accurate on average than EururRM-7B on the MATH dataset, with EururRM-7B being a state-of-the-art open-source reward model on the math and code domains [53]. A notable observation is that while many baseline reward models perform well on specific subsets of the data, SWIFT demonstrates consistent and robust performance across the entire dataset. Although the fine-tuned EururRM-7B achieves respectable results, it still falls short of SWIFT in accuracy and incurs substantially higher computational costs during both training and inference. In contrast, SWIFT not only achieves the best overall performance but does so with a parameter footprint of less than 0.005% of the baselines and is trained on only 6,000 samples per dataset (see Section 5.3 for

detailed comparison). This highlights the exceptional efficiency of our approach.

It is noteworthy that the results presented here were achieved **without extensive hyperparameter tuning**. Further gains could likely be realized with more refined optimization, increasing the training dataset size (Section 5.6), or training SWIFT on a subset of layers (Section 5.7).

5.3 Efficiency Analysis

We demonstrate SWIFT’s significant parameter efficiency and data efficiency in Table 3. Moreover, we evaluate the computational efficiency of SWIFT by comparing the average time and FLOPs required to assign a reward to each sample. Figure 3 presents a comparative analysis of SWIFT and baseline reward models across various datasets and task-performing models. The results show that SWIFT achieves orders-of-magnitude improvements in both time and FLOPs compared to the baselines. Notably, as shown in Section 5.6, even a significantly smaller training dataset leads to satisfactory performance. This substantial reduction in computational cost, combined with its parameter efficiency and data efficiency, makes SWIFT a highly practical solution for reward modeling in both resource-constrained environments and large-scale applications.

In addition to per-sample scoring efficiency, we also measure end-to-end BoN inference time on CoinFlip. The proportion of time spent by the task-performing LLM to generate answers ranges from 14.7% (Llama3-3B+Starling) to 59.3% (Ministral-8B+RLHFlow), with the remainder dominated by reward-model scoring. Replacing the baseline RM with SWIFT yields $1.7\times$ – $6.7\times$ overall speed-up (specifically, Llama3-3B+Starling: $0.3649s \rightarrow 0.0543s$, $6.7\times$ faster; Ministral-8B+RLHFlow: $0.1762s \rightarrow 0.1051s$, $1.7\times$). Notably, even the smallest speed-up ($1.7\times$) reflects SWIFT’s overhead being negligible compared to text-based baseline scoring, and the largest gain ($6.7\times$) demonstrates that SWIFT can reduce wall-clock time by an order of magnitude when the baseline RM is particularly heavy. SWIFT’s advantage extends beyond inference efficiency: it also significantly reduces training costs and GPU memory consumption during both training and deployment due to its minimal size.

5.4 Generalization Test

To assess transferability, we pretrain SWIFT on 10k samples of DeepScaleR dataset [28] and evaluate it directly on downstream benchmarks (no dataset-specific fine-tuning). With Ministral-8B and BoN@64, SWIFT achieves 62.8/93.6/75.8 on MATH/GSM8K/AQuA_RAT, consistently exceeding all baselines (e.g., fine-tuned Eurur: 57.2/92.8/71.0; Skywork: 61.6/87.4/72.0) and achieving performance comparable to fine-tuned SWIFT. In addition, to further validate that the strong performance of SWIFT does not benefit from fine-tuning solely, we fine-tune Skywork-Llama3.1-8B (the strongest baseline RM on MATH) and evaluate BoN@64 on MATH. Across task LLMs Llama3-3B/Llama3-8B/Ministral-8B, fine-tuned Skywork yields 52.2/54.0/62.6, while SWIFT achieves 53.6/62.6/62.8, consistently outperforming the fine-tuned baseline. This demonstrates that SWIFT’s improvements stem from its design rather than fine-tuning alone, and confirms robust cross-dataset generalization.

Table 1: Different methods’ best-of-N sampling performance on mathematics domain, including MATH, GSM8K, and AQuA_RAT datasets. Results represent accuracy (percentages omitted). Bold: best performance; Underline: second-best performance; @k: indicates the number of reasoning paths used in best-of-N sampling.

MATH Dataset										
Reward Model	Llama-3.2-3B BoN@1: 39.0			Llama-3.1-8B BoN@1: 47.2			Ministral-8B BoN@1: 51.0			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
Eurus-7B [53]	42.6	48.2	46.8	50.8	52.0	52.2	54.6	56.8	55.0	51.0
Skywork-Llama3.1-8B [24]	43.8	<u>48.4</u>	<u>48.8</u>	52.2	52.4	<u>53.4</u>	<u>56.8</u>	<u>59.0</u>	<u>61.6</u>	<u>52.9</u>
Starling-7B [59]	39.6	41.2	39.8	50.4	49.0	49.0	<u>53.8</u>	50.2	47.0	46.7
Ultra-13B [8]	<u>44.6</u>	47.4	44.4	<u>53.0</u>	50.6	50.4	53.6	53.0	54.0	50.1
RLHFlow-8B-Deepseek [49]	43.2	46.4	47.6	51.2	49.6	49.8	56.2	58.4	57.8	51.1
Math-Shepherd-7B [44]	43.8	42.6	43.6	48.8	50.8	49.0	55.8	58.8	54.8	49.8
Fine-tuned Eurus-7B	42.2	47.0	46.6	51.8	<u>53.0</u>	50.2	54.8	57.2	57.2	51.1
SWIFT (ours)	49.8	54.6	53.6	55.4	59.4	62.6	57.8	61.6	62.8	57.5
GSM8K Dataset										
Reward Model	Llama-3.2-3B BoN@1: 53.0			Llama-3.1-8B BoN@1: 80.4			Ministral-8B BoN@1: 79.6			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
Eurus-7B [53]	69.8	76.2	82.4	87.0	89.0	90.4	87.0	90.8	90.4	84.8
Skywork-Llama3.1-8B [24]	77.2	81.8	85.2	89.6	89.4	89.2	83.0	85.8	87.4	85.4
Starling-7B [59]	62.4	65.6	71.4	83.0	87.2	85.4	78.4	81.2	80.0	77.2
Ultra-13B [8]	72.6	78.6	82.4	88.0	88.6	86.8	82.6	84.6	82.0	82.9
RLHFlow-8B-Deepseek [49]	52.4	53.8	60.0	80.4	77.0	80.0	<u>88.4</u>	91.2	93.4	75.2
Math-Shepherd-7B [44]	54.8	49.8	43.4	80.6	75.4	73.0	84.8	86.8	86.8	70.6
Fine-tuned Eurus-7B	<u>79.2</u>	<u>84.4</u>	<u>86.8</u>	<u>90.0</u>	91.2	91.0	88.2	<u>91.8</u>	92.8	<u>88.4</u>
SWIFT (ours)	80.0	86.4	87.6	90.6	<u>89.6</u>	<u>90.6</u>	89.8	93.2	93.4	89.0
AQuA_RAT Dataset										
Reward Model	Llama-3.2-3B BoN@1: 42.0			Llama-3.1-8B BoN@1: 53.6			Ministral-8B BoN@1: 56.6			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
Eurus-7B [53]	44.6	42.8	44.6	58.6	60.6	56.8	48.4	34.6	27.2	46.5
Skywork-Llama3.1-8B [24]	<u>60.4</u>	<u>68.0</u>	<u>69.4</u>	<u>69.0</u>	77.0	79.2	65.6	71.2	<u>72.0</u>	<u>70.2</u>
Starling-7B [59]	51.4	55.0	56.6	64.2	70.8	69.8	56.4	48.6	42.8	57.3
Ultra-13B [8]	58.4	61.6	64.8	67.4	<u>72.8</u>	71.8	58.4	51.4	48.0	61.6
RLHFlow-8B-Deepseek [49]	53.8	54.0	48.8	63.6	65.0	61.4	64.6	63.8	62.6	59.7
Math-Shepherd-7B [44]	52.2	53.4	53.0	64.0	67.8	66.2	63.6	60.4	60.2	60.1
Fine-tuned Eurus-7B	57.4	66.6	62.0	68.8	<u>72.8</u>	69.8	<u>71.2</u>	<u>71.8</u>	71.0	67.9
SWIFT (ours)	61.0	70.4	70.8	69.6	77.0	<u>77.0</u>	75.2	75.8	78.0	72.8

5.5 Domains beyond reasoning accuracy (helpfulness/safety)

Finally, to test whether SWIFT extends to alignment-oriented evaluation, we run Best-of-N on PKU-SafeRLHF dataset [19] using GPT-4o as judge (Ministral-8B; BoN@4). The result is characterized in Table 4. SWIFT significantly outperforms all large text-based reward models. The consistent helpfulness and safety improvement over

these baselines suggest that SWIFT’s hidden-state representations capture nuanced response quality beyond reasoning correctness, making it suitable for general alignment tasks.

5.6 Scalability with Training Set Size and Reasoning Paths

Scaling large language models (LLMs) can be achieved through various strategies, broadly categorized as training-time scaling and

Table 2: Different methods’ best-of-N sampling performance on general domain, including Imbue Code Comprehension, HellaSwag and CoinFlip datasets. Results represent accuracy (percentages omitted). Bold: best performance; Underline: second-best performance; @k: indicates the number of reasoning paths used in best-of-N sampling.

Imbue Code Comprehension Dataset										
Reward Model	Llama-3.2-3B BoN@1: 17.2			Llama-3.1-8B BoN@1: 47.6			Ministral-8B BoN@1: 54.4			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
Eurus-7B [53]	26.0	28.0	30.2	60.0	62.2	66.2	59.0	61.6	64.0	50.8
Skywork-Llama3.1-8B [24]	32.6	47.4	56.0	<u>67.0</u>	70.8	75.2	66.6	71.4	74.8	62.4
Starling-7B [59]	18.8	21.0	21.0	57.6	62.6	64.2	58.6	60.4	62.6	47.4
Ultra-13B [8]	28.0	34.8	38.8	61.8	66.6	68.6	60.6	64.4	65.2	54.3
RLHFlow-8B-Deepseek [49]	29.2	37.0	44.4	66.4	<u>71.4</u>	72.0	<u>68.4</u>	<u>74.0</u>	<u>75.2</u>	59.8
Math-Shepherd-7B [44]	22.6	25.4	25.4	60.6	60.0	55.8	63.0	65.4	61.4	48.8
Fine-tuned Eurus-7B	<u>35.6</u>	<u>54.4</u>	<u>60.4</u>	<u>67.0</u>	69.4	<u>76.0</u>	67.8	71.0	71.8	<u>63.7</u>
SWIFT (ours)	36.8	57.6	65.6	74.0	84.0	85.0	74.6	82.4	85.4	71.7
HellaSwag Dataset										
Reward Model	Llama-3.2-3B BoN@1: 55.2			Llama-3.1-8B BoN@1: 61.2			Ministral-8B BoN@1: 68.2			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
Eurus-7B [53]	67.4	71.6	72.0	72.6	73.6	74.0	77.8	77.4	78.0	73.8
Skywork-Llama3.1-8B [24]	63.0	64.0	67.0	70.6	71.6	71.8	73.0	74.0	76.4	70.2
Starling-7B [59]	53.6	56.2	59.0	64.8	65.8	63.6	66.0	65.0	63.6	62.0
Ultra-13B [8]	60.8	64.0	64.2	69.6	69.4	68.2	67.8	67.8	70.6	66.9
RLHFlow-8B-Deepseek [49]	65.2	68.4	69.2	68.2	70.8	71.0	75.2	78.2	79.8	71.8
Math-Shepherd-7B [44]	59.8	61.0	62.2	66.0	65.2	63.8	70.2	69.8	73.0	65.7
Fine-tuned Eurus-7B	<u>70.4</u>	76.2	77.4	<u>75.2</u>	<u>79.4</u>	<u>78.6</u>	<u>80.4</u>	<u>81.4</u>	<u>81.6</u>	<u>77.8</u>
SWIFT (ours)	71.0	<u>75.8</u>	77.4	78.0	82.4	83.2	83.4	88.0	88.2	80.8
CoinFlip Dataset										
Reward Model	Llama-3.2-3B BoN@1: 56.2			Llama-3.1-8B BoN@1: 62.6			Ministral-8B BoN@1: 20.0			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
Eurus-7B [53]	65.6	64.2	64.6	83.6	85.6	91.8	45.8	69.8	77.4	72.0
Skywork-Llama3.1-8B [24]	72.6	81.0	88.0	88.0	94.4	97.4	25.2	28.2	26.0	66.8
Starling-7B [59]	66.6	73.0	76.2	79.0	84.2	87.2	22.0	19.6	21.0	58.8
Ultra-13B [8]	61.6	62.0	59.4	82.0	85.6	84.0	14.0	11.4	11.4	52.4
RLHFlow-8B-Deepseek [49]	64.0	65.6	75.6	83.4	96.4	99.4	34.6	49.4	72.8	71.2
Math-Shepherd-7B [44]	55.8	46.4	35.4	62.4	40.4	15.2	20.0	19.0	15.4	34.4
Fine-tuned Eurus-7B	<u>91.2</u>	<u>99.4</u>	<u>99.8</u>	<u>95.2</u>	99.8	100.0	57.8	<u>95.8</u>	99.8	<u>93.2</u>
SWIFT (ours)	91.6	99.8	100.0	95.4	<u>99.6</u>	100.0	57.8	96.2	<u>99.6</u>	93.3

test-time scaling [20, 43]. In this section, we investigate the scalability of SWIFT with respect to both training set size and the number of reasoning paths used during inference.

From Figure 4, our results demonstrate that SWIFT’s accuracy consistently improves with both the number of training samples and the size of the reasoning paths set. Specifically, under different BoN@k settings, we find that SWIFT’s accuracy increases monotonically with training set size. Likewise, for any given training set

size, SWIFT’s accuracy also improves as the number of reasoning paths used in the BoN sampling process increases.

It is interesting to note that, as shown in Figure 4, the performance of SWIFT with training-time scaling does not appear to plateau even with 6000 training samples, which is the one used in the experiments of Section 5.2. This suggests that further increasing the training set size may yield additional performance improvements, underscoring the scalability of SWIFT. These results

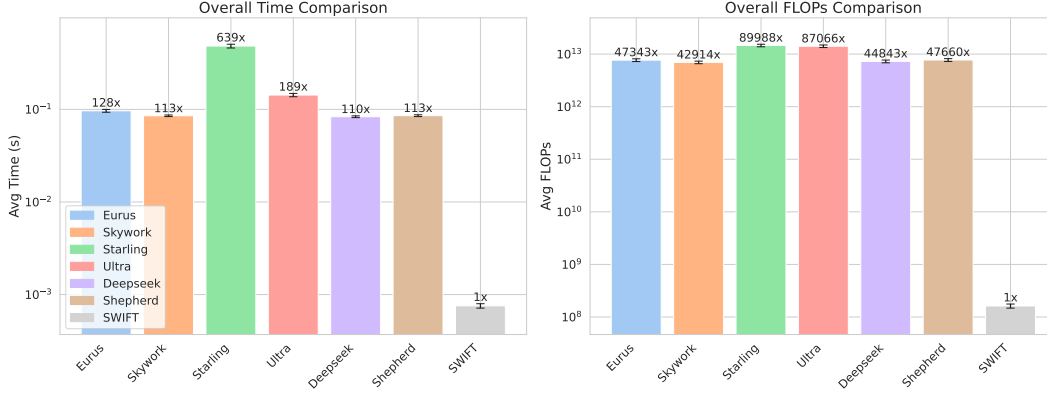


Figure 3: Comparison of average time and FLOPs per sample for SWIFT and baselines, averaged across different datasets and task-performing models with 1-sigma error bars using standard error of the mean. The y-axis is plotted in log-scale. It shows that SWIFT achieves orders-of-magnitude higher efficiency than baselines.

Table 3: Comparison of Reward Model Parameters and Training Data Size (number of questions $\times$ number of answers). Our method is remarkably efficient in terms of parameters and data.

Reward Model	Parameters	Training data size
Eurus-7B	7.1×10^9	$5.8 \times 10^5 \times 2$
Skywork-Llama3.1-8B	7.5×10^9	$8.0 \times 10^4 \times 2$
Starling-7B	6.7×10^9	$1.8 \times 10^5 \times 7$
Ultra-13B	1.3×10^{10}	$7.5 \times 10^5 \times 2$
RLHFlow-8B-Deepseek	8.0×10^9	$2.5 \times 10^5 \times 1$
Math-Shepherd-7B	7.2×10^9	$4.5 \times 10^5 \times 1$
SWIFT (on Llama-3.2-3B)	1.8×10^5	$6.0 \times 10^3 \times 8$
SWIFT (on Llama-3.1-8B)	2.7×10^5	$6.0 \times 10^3 \times 8$
SWIFT (on Ministral-8B)	3.0×10^5	$6.0 \times 10^3 \times 8$

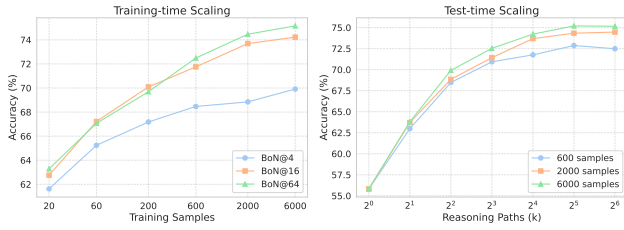


Figure 4: SWIFT has positive scaling with increased number of training samples and with the number of reasoning paths for inference. The result is averaged across datasets and task-performing models.

highlight SWIFT as a compelling and practical approach to scaling reward modeling for real-world applications.

5.7 Maintaining Performance with Selective Layer Utilization for Higher Efficiency

While SWIFT leverages hidden states from all layers of the LLM, we investigated the potential for further parameter reduction by selectively using hidden states from only a subset of layers. Specifically, for the Llama-3.1-8B model, which comprises 32 layers (plus an embedding layer), we explored training SWIFT using only the hidden states from layers 16, 24, 28, and 32, following the setting in Azaria and Mitchell [2]. This configuration reduces the parameter count to approximately 4/33 of the full-layer SWIFT model. The results, presented in Table 5, demonstrate that performance remains comparable to the full-layer SWIFT, with certain settings even showing improved accuracy. Notably, on the GSM8K dataset, the BoN@16 and BoN@64 accuracies surpass those of the full-layer SWIFT. As shown in Table 1, the performance of the layer-selected SWIFT also **consistently exceeds that of baselines** with significantly larger parameter counts, further underscoring the efficiency and effectiveness of our approach.

We further find that **later layers encode stronger correctness signals**: using only the *first* 4 layers gives BoN@64 accuracy 38.2/44.6/45.4 (Llama3-3B/Llama3-8B/Ministral-8B on MATH), while using only the *last* 4 layers improves it to 55.4/60.4/61.8.

5.8 Logit-Based Training for Closed-Source LLMs

While SWIFT offers significant advantages in terms of efficiency and performance, making it well-suited for deployment by large model providers and end-users, a limitation arises when working with closed-source LLMs where hidden states are inaccessible. To overcome this constraint, we leverage the fact that some closed-source models still provide access to their logits, such as GPT-3.5-turbo [34] and GPT-4 [1], as demonstrated by Finlayson et al. [11] and OpenAI [35]. We train SWIFT directly on these logits, resulting in a model with a parameter count that is orders of magnitude smaller than traditional reward models - and even smaller than SWIFT trained on all layers. This enables seamless deployment on

Table 4: Alignment performance on the PKU-SafeRLHF dataset evaluated by GPT-4o. We report BoN@4 accuracy using Ministral-8B as the policy. SWIFT consistently outperforms strong baselines on both helpfulness and safety dimensions.

Dimension	SWIFT	Eurus	Skywork	Ultra	RLHFlow	Starling	Shepherd
Helpfulness	64.2	59.8	63.2	54.6	61.0	60.4	53.0
Safety	92.2	89.8	91.6	79.0	90.2	91.6	84.2

Table 5: SWIFT’s high Best-of-N sampling performance is maintained even when trained only on layers 16, 24, 28, and 32 for Llama-3.1-8B, leading to a substantial reduction in parameters and improved efficiency.

Method	MATH BoN@1: 47.2			GSM8K BoN@1: 80.4			AQuA_RAT BoN@1: 53.6			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
SWIFT on partial layers	55.8	59.0	60.0	89.4	90.8	92.0	69.6	76.8	77.4	74.5
SWIFT on all layers	55.4	59.4	62.6	90.6	89.6	90.6	69.6	77.0	77.0	74.6

Table 6: Training SWIFT solely on logits can also yield high performance and can outperform many baselines with orders-of-magnitude higher efficiency. Results on GSM8K and AQuA_RAT dataset can be found in the full version.

MATH Dataset										
Method	Llama-3.2-3B BoN@1: 39.0			Llama-3.1-8B BoN@1: 47.2			Ministral-8B BoN@1: 51.0			Avg.
	@4	@16	@64	@4	@16	@64	@4	@16	@64	
SWIFT on logits	44.6	49.6	50.6	54.2	54.4	55.8	53.4	55.6	57.4	52.8

personal devices such as smartphones and personal computers. As demonstrated in Table 6, the logit-based SWIFT maintains strong performance. Furthermore, as shown in Table 1, its performance even surpasses that of many baselines. These results highlight the versatility and adaptability of SWIFT, making it a viable solution even when access to hidden states is restricted.

Concretely, with BoN@64, logit-only SWIFT reaches 50.6/55.8/57.4 on MATH, 81.0/91.2/90.2 on GSM8K, and 65.2/78.0/74.0 on AQuA_RAT for Llama-3.2-3B/Llama-3.1-8B/Ministral-8B, respectively, demonstrating that informative correctness signals can be extracted even from logit access alone.

5.9 Combining SWIFT with External Reward Models

SWIFT offers a highly parameter-efficient approach to reward modeling, enabling its seamless integration with existing external reward models. Given SWIFT’s minimal parameter footprint (less than 0.005% of baseline models), the additional computational overhead introduced by incorporating SWIFT is negligible compared to relying solely on the external reward model. We explore two straightforward methods for combining SWIFT with external rewards: rank selection and scaled averaging. In rank selection, we choose the sample with the best rank as determined by either SWIFT or the external reward model. In scaled averaging, we compute the

average of the rewards from both models after normalizing them to the $[0, 1]$ range.

We evaluated the effectiveness of these combination strategies by integrating SWIFT with various traditional reward models across different datasets and task-performing models, assessing performance using best-of-N sampling. Figure 5 presents the average results across different datasets for both ranking and scaling methods, applied to various task-performing models. Our findings indicate that combining SWIFT with external reward models can lead to further improvements in accuracy, particularly on the GSM8K and AQuA-RAT datasets. The MATH dataset did not exhibit significant performance gains, potentially because SWIFT already significantly outperforms the external reward models on this benchmark. This performance boost is likely due to SWIFT and external reward models capturing different types of errors, resulting in improved overall accuracy-particularly with a higher number of best-of-N reasoning paths. This combination strategy offers a pathway to further performance gains without incurring significant computational overhead.

5.10 Analysis on SWIFT

To investigate the mechanism of SWIFT, we present illustrative examples of its scoring of reasoning paths in appendix of the full version. The examples with the highest and lowest gating $\times$ reward values are highlighted, along with their corresponding numerical

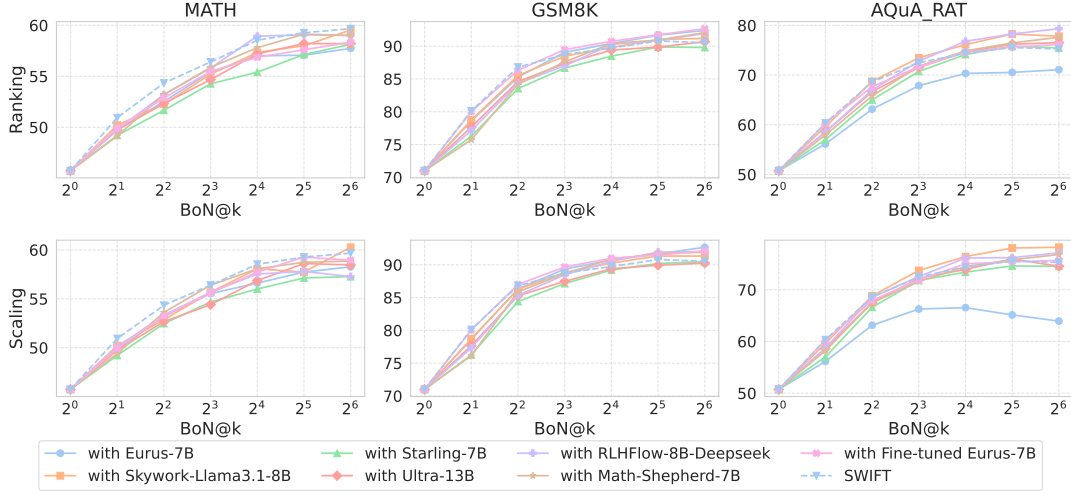


Figure 5: Average performance of combined SWIFT and external reward models across different datasets and task-performing models, for both rank selection and scaled averaging.

scores. Notably, SWIFT assigns relatively high or low gating $\times$ reward values to specific numerical and mathematical symbols, as well as particular tokens such as “answer”, “boxed” and the special token ‘<|eot_id|>’. This behavior suggests that SWIFT possesses the ability to identify and weigh key components within the reasoning process.

In addition, we also conduct ablation study on the gating mechanism of SWIFT. SWIFT includes a lightweight token-level gating module that re-weights token representations before aggregating them into a path-level score. We ablate this component by removing gating while keeping the same training budget and evaluation protocol. On MATH, gating yields consistent gains across all BoN settings. For BoN@64, accuracies improve from 49.4 to 53.6 (Llama-3.2-3B), 57.6 to 62.6 (Llama-3.1-8B), and 61.4 to 62.8 (Minstral-8B). Similar trends hold at BoN@4 and BoN@16, confirming that gating is beneficial across varying numbers of candidate paths. This token-level selection mechanism allows SWIFT to emphasize semantically decisive portions of reasoning traces, such as intermediate steps, numerical computations, and conclusion markers, while down-weighting boilerplate text. The full result can be found in the full version.

To better understand where SWIFT’s gains come from, we further break down improvements by MATH sub-domain (Minstral-8B as the task LLM; BoN@64). SWIFT improves over Math-Shepherd-7B across all sub-domains, with gains ranging from +2.44 points (Geometry) to +10.52 points (Counting & Probability), and consistently strong gains on Algebra (+8.87), Intermediate Algebra (+10.31), and Number Theory (+9.68). See Table 7 for the full breakdown.

We also compare alternative objectives for training SWIFT on MATH, including hinge-style and preference-style losses. Across three task LLMs (Llama3-3B/Llama3-8B/Minstral-8B) and three BoN settings (@4, @16, @64), cross-entropy achieves the best average accuracy (57.5), followed by DPO (57.0), hinge loss (56.8), InfoNCA (55.1) and NCA (54.4). The relatively small gap between the top three suggests that SWIFT is robust to the choice of training

signal. Cross-entropy remains our default due to its consistent edge and simpler implementation.

The detailed analysis for SWIFT can be found in the full version.

6 Conclusion

In this paper, we presented SWIFT, a novel reward modeling approach for LLMs that leverages the rich information contained in their internal hidden states. SWIFT offers both high efficiency and effectiveness, and it scales well during both training and inference. We further showed that comparable performance can be achieved by training SWIFT on a subset of layers or even on logits alone, significantly reducing computational costs and making it more versatile. The successful integration of SWIFT with standard reward models underscores its flexibility and potential.

Admittedly, this work has some limitations. SWIFT’s reliance on access to the LLM’s hidden states or logits may reduce its applicability to commercial LLMs that do not expose this information to users. However, its efficiency and ease of deployment make SWIFT especially well-suited to LLM providers themselves. In addition, SWIFT must be trained separately for each LLM as hidden-state dimensionality and layer count vary across models. Nonetheless, given its small model size and minimal data requirements, the associated computational overhead is negligible, especially when compared to the cost of training the task-performing LLM itself.

Acknowledgments

This work was supported by the National Science Foundation (NSF) under Grant No. III-2106758.

We gratefully acknowledge Mrs. Jiahe Jin from Zhiyuan College, Shanghai Jiao Tong University, for her invaluable assistance in creating the initial version of our illustrations. We also thank Mr. Zhihui Xie from The University of Hong Kong for offering precious suggestions.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Amos Azaria and Tom Mitchell. 2023. The Internal State of an LLM Knows When It's Lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 967–976. doi:10.18653/v1/2023.findings-emnlp.68
- [3] Suresh Balakrishnama and Aravind Ganapathiraju. 1998. Linear discriminant analysis-a brief tutorial. *Institute for Signal and information Processing* 18, 1998 (1998), 1–8.
- [4] Fazl Barez, Tingchen Fu, Ameiya Prabhu, Stephen Casper, Amartya Sanyal, Adel Bibi, Aidan O'Gara, Robert Kirk, Ben Bucknall, Tim Fiste, et al. 2025. Open problems in machine unlearning for ai safety. *arXiv preprint arXiv:2501.04952* (2025).
- [5] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. 2023. Discovering Latent Knowledge in Language Models Without Supervision. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=ETKGubY0hcs>
- [6] Alex J Chan, Hao Sun, Samuel Holt, and Mihaela Van Der Schaar. 2024. Dense reward for free in reinforcement learning from human feedback. *arXiv preprint arXiv:2402.00782* (2024).
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
- [8] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. ULTRAfeedback: Boosting Language Models with Scaled AI Feedback. In *Forty-first International Conference on Machine Learning*. <https://openreview.net/forum?id=BOOrDpKHjI>
- [9] Sayantan Dasgupta and Trevor Cohn. 2025. Improving Language Model Distillation through Hidden State Matching. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=IcVSKhVpKu>
- [10] Javier Ferrando, Gabriele Sarti, Arianna Bisazza, and Marta R Costa-Jussa. 2024. A primer on the inner workings of transformer-based language models. *arXiv preprint arXiv:2405.00208* (2024).
- [11] Matthew Finlayson, Xiang Ren, and Swabha Swayamdipta. 2024. Logits of API-Protected LLMs Leak Proprietary Information. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=oRcYFm8vyB>
- [12] Adam Fisch, Jacob Eisenstein, Vicky Zayats, Alekh Agarwal, Ahmad Beirami, Chirag Nagpal, Pete Shaw, and Jonathan Berant. 2024. Robust preference optimization through reward model distillation. *arXiv preprint arXiv:2405.19316* (2024).
- [13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [14] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirog Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [15] Jizhou Guo, Zhaomin Wu, Hanchen Yang, and Philip S. Yu. 2025. Mining Intrinsic Rewards from LLM Hidden States for Efficient Best-of-N Sampling. *arXiv:2505.12225 [cs.LG]* <https://arxiv.org/abs/2505.12225>
- [16] Jun Han and Claudio Moraga. 1995. The influence of the sigmoid function parameters on the speed of backpropagation learning. In *International workshop on artificial neural networks*. Springer, 195–201.
- [17] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. <https://openreview.net/forum?id=7Bywt2mQsCe>
- [18] Imbue Team. 2024. Sanitized open-source datasets for natural language and code understanding: how we evaluated our 70B model. <https://imbue.com/research/70b-evals/>.
- [19] Jiaming Ji, Donghai Hong, Borong Zhang, Boyuan Chen, Josef Dai, Boren Zheng, Tianyi Alex Qiu, Jiayi Zhou, Kaile Wang, Boxun Li, et al. 2025. Pku-saferlhf: Towards multi-level safety alignment for llms with human preference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 31983–32016.
- [20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [21] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, Lester James Validad Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2025. RewardBench: Evaluating Reward Models for Language Modeling. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 1755–1797. <https://aclanthology.org/2025.findings-naacl.96/>
- [22] Kenneth Li, Aspen K Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. 2023. Emergent world representations: Exploring a sequence model trained on a synthetic task. *ICLR* (2023).
- [23] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 158–167. doi:10.18653/v1/P17-1015
- [24] Chris Yuhao Liu, Liang Zeng, Jiakai Liu, Rui Yan, Jujie He, Chaojie Wang, Shuicheng Yan, Yang Liu, and Yahui Zhou. 2024. Skywork-reward: Bag of tricks for reward modeling in llms. *arXiv preprint arXiv:2410.18451* (2024).
- [25] Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu Li, Biqing Qi, Wanli Ouyang, and Bowen Zhou. 2025. Can 1B LLM Surpass 405B LLM? Rethinking Compute-Optimal Test-Time Scaling. *arXiv preprint arXiv:2502.06703* (2025).
- [26] Xiao Liu, Hanyu Lai, Hao Yu, Yifan Xu, Aohan Zeng, Zhengxiao Du, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023. WebGLM: towards an efficient web-enhanced question answering system with human preferences. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 4549–4560.
- [27] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Bkg6RiCqY7>
- [28] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. 2025. DeepScaleR: Surpassing O1-Preview with a 1.5B Model by Scaling RL. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>. Notion Blog.
- [29] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in gpt. *Advances in neural information processing systems* 35 (2022), 17359–17372.
- [30] Meta. 2024. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
- [31] Mistral AI. 2024. Introducing the world's best edge models. <https://mistral.ai/news/ministraux>.
- [32] Fengran Mo, Jian-Yun Nie, Kaiyu Huang, Kelong Mao, Yutao Zhu, Peng Li, and Yang Liu. 2023. Learning to relate to previous turns in conversational search. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1722–1732.
- [33] Hyuk Namgoong, Jeosu Jung, Sangkeun Jung, and Yoonhyung Roh. 2024. Exploring Domain Robust Lightweight Reward Models based on Router Mechanism. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 8644–8652. doi:10.18653/v1/2024.findings-acl.511
- [34] OpenAI. 2023. GPT-3.5 Turbo. <https://platform.openai.com/docs/models/gpt-3.5-turbo>.
- [35] OpenAI. 2023. Using logprobs. https://cookbook.openai.com/examples/using_logprobs.
- [36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [37] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *the journal of machine Learning research* 12 (2011), 2825–2830.
- [38] Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. 2024. From \$r\$ to \$Q^*\$: Your Language Model is Secretly a Q-Function. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=kEVcNxtqXk>
- [39] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2023), 53728–53741.
- [40] Lorenzo Rosasco, Ernesto De Vito, Andrea Caponnetto, Michele Piana, and Alessandro Verri. 2004. Are loss functions all the same? *Neural computation* 16, 5 (2004), 1063–1076.
- [41] Karan Samel, Cheng Li, Weize Kong, Tao Chen, Mingyang Zhang, Shaleen Gupta, Swaraj Khadanga, Wensong Xu, Xingyu Wang, Kashyap Kolipaka, et al. 2023. End-to-End Query Term Weighting. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4778–4786.
- [42] Tao Shen, Xiubo Geng, Chongyang Tao, Can Xu, Guodong Long, Kai Zhang, and Daxin Jiang. 2023. Unifier: A unified retriever for large-scale retrieval. In

Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 4787–4799.

- [43] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=4FWAwZtd2n>
- [44] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 9426–9439. doi:10.18653/v1/2024.acl-long.510
- [45] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [46] Svante Wold, Kim Esbensen, and Paul Geladi. 1987. Principal component analysis. *Chemometrics and intelligent laboratory systems* 2, 1-3 (1987), 37–52.
- [47] Zhaomin Wu, Jizhou Guo, Junyi Hou, Bingsheng He, Lixin Fan, and Qiang Yang. 2024. Model-Based Differentially Private Knowledge Transfer for Large Language Models. *arXiv preprint arXiv:2410.10481* (2024).
- [48] Zhihui Xie, Jizhou Guo, Tong Yu, and Shuai Li. 2024. Calibrating Reasoning in Language Models with Internal Consistency. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=udZKVMPf3S>
- [49] Wei Xiong, Hanning Zhang, Nan Jiang, and Tong Zhang. 2024. An Implementation of Generative PRM. <https://github.com/RLHFlow/RLHF-Reward-Modeling>.
- [50] Shicheng Xu, Liang Pang, Huawei Shen, Xueqi Cheng, and Tat-Seng Chua. 2024. Search-in-the-chain: Interactively enhancing large language models with search for knowledge-intensive tasks. In *Proceedings of the ACM Web Conference 2024*. 1362–1373.
- [51] Yuan Yao, Lorenzo Rosasco, and Andrea Caporinnetto. 2007. On early stopping in gradient descent learning. *Constructive approximation* 26, 2 (2007), 289–315.
- [52] Qifan Yu, Wei Chow, Zhongqi Yue, Kaihang Pan, Yang Wu, Xiaoyang Wan, Juncheng Li, Siliang Tang, Hanwang Zhang, and Yueting Zhuang. 2024. AnyEdit: Mastering Unified High-Quality Image Editing for Any Idea. *arXiv preprint arXiv:2411.15738* (2024).
- [53] Lifan Yuan, Ganqu Cui, Hanbin Wang, Ning Ding, Xingyao Wang, Boji Shan, Zeyuan Liu, Jia Deng, Huimin Chen, Ruobing Xie, Yankai Lin, Zhenghao Liu, Bowen Zhou, Hao Peng, Zhiyuan Liu, and Maosong Sun. 2025. Advancing LLM Reasoning Generalists with Preference Trees. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=2ea5TNVR0c>
- [54] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.
- [55] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. 2025. Reasoning Models Know When They’re Right: Probing Hidden States for Self-Verification. *arXiv preprint arXiv:2504.05419* (2025).
- [56] Jing Zhang, Xiaokang Zhang, Daniel Zhang-Li, Jifan Yu, Zijun Yao, Zeyao Ma, Yiqi Xu, Haohua Wang, Xiaohan Zhang, Nianyi Lin, et al. 2023. Glm-dialog: Noise-tolerant pre-training for knowledge-grounded dialogue generation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5564–5575.
- [57] Xiaoying Zhang, Jean-Francois Ton, Wei Shen, Hongning Wang, and Yang Liu. 2024. Mitigating Reward Overoptimization via Lightweight Uncertainty Estimation. *Advances in Neural Information Processing Systems* 37 (2024), 81717–81747.
- [58] Ruiwen Zhou, Yingxuan Yang, Muning Wen, Ying Wen, Wenhao Wang, Chunling Xi, Guoqiang Xu, Yong Yu, and Weinan Zhang. 2024. Trad: Enhancing llm agents with step-wise thought retrieval and aligned decision. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3–13.
- [59] Banghua Zhu, Evan Frick, Tianhao Wu, Hanlin Zhu, Karthik Ganesan, Wei-Lin Chiang, Jian Zhang, and Jiantao Jiao. 2024. Starling-7B: Improving Helpfulness and Harmlessness with RLAI. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=GqDntYTTbk>
- [60] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. 2023. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405* (2023).

Appendix

Due to the strict page limits of the conference proceedings (main text ≤ 9 pages; total ≤ 12 pages), we omit extended appendix material in this camera-ready version. The full version (including extended implementation details, ablations, additional figures/tables, and case studies) is available at <https://arxiv.org/abs/2505.12225>.

A MATH Sub-domain Breakdown

To better understand when SWIFT helps, we compare SWIFT against Math-Shepherd-7B on MATH and break down the BoN@64 gains by sub-domain (Minstral-8B as the task LLM). Table 7 shows improvements across all sub-domains, with larger gains on categories that typically require more multi-step reasoning.

Table 7: SWIFT gains over Math-Shepherd-7B by MATH sub-domain (Minstral-8B, BoN@64; accuracy difference in percentage points).

Category	Gain
Precalculus	7.14
Algebra	8.87
Intermediate Algebra	10.31
Number Theory	9.68
Prealgebra	4.88
Geometry	2.44
Counting & Probability	10.52

B Implementation Details

This appendix section provides additional details on the experimental setup and training procedures used in our work. Additional details can be found in the full version.

B.1 Hardware and Software

We conduct our experiment in a system with 8x NVIDIA A100 GPUs (80GB each) and an Intel(R) Xeon(R) Gold 6346 CPU @ 3.10GHz with 1.0TB of CPU memory (large CPU memory is unnecessary). We used the PyTorch framework [36] for implementing our models and training procedures.

B.2 Details of the Preliminary Experiments

Our pipeline consists of the following steps:

- (1) **Hidden State Extraction:** We use the Llama-3.1-8B-Instruct model [13] and extract the hidden states h_t^l for each token t at each layer l during the generation of reasoning paths on the MATH dataset [17].
- (2) **Averaging:** For each reasoning path, we average the hidden states across all tokens within that path to obtain a single vector representation $\bar{h}^l$ for each layer l .
- (3) **Dimensionality Reduction (PCA):** We apply Principal Component Analysis (PCA) [46] to reduce the dimensionality of $\bar{h}^l$ to 50 dimensions. This step helps to mitigate overfitting that we observed with direct application of LDA.
- (4) **Classification (LDA):** We train a Linear Discriminant Analysis (LDA) classifier [3] on the reduced representations to predict whether the reasoning path is correct or incorrect.

The entire pipeline is *linear* to validate that hidden states of the reasoning paths encode a linear representation. The PCA+LDA model was trained using the scikit-learn library [37]. We adopted the default settings in scikit-learn.

B.3 SWIFT Training Details

The SWIFT model was trained using the AdamW optimizer [27] with the following hyperparameters:

- **Learning Rate:** 1×10^{-4}
- **Weight Decay:** 1×10^{-5}
- **Batch Size:** 16

For each dataset, we created a training set of 6000 instances, drawn from the original training set, and a held-out test set of 500 instances, drawn from the original validation set (due to the absence of labels in some datasets' test sets). This consistent data preparation approach was applied across all datasets, with the exceptions of GSM8K (where the original test set was used due to the lack of a validation set) and Imbue Code Comprehension (where we performed a manual train/test split as no split was provided).

We employed early stopping [51] to prevent overfitting. We further split the training data into training and validation sets using an 80/20 ratio. After each training epoch, we evaluated the model's performance on the validation set. We used the cross-entropy with logits loss on validation set as the criterion for early stopping. We used a patience of 3 epochs, meaning that training was terminated if the validation loss did not improve for 3 consecutive epochs. The model with the lowest validation loss across all epochs was selected as the final SWIFT model.

The full training details (including evaluation and generation details) can be found in the appendix of the full version.

C Baseline Reward Models

Our experiments compare against six baseline reward models: **EurusRM-7B** [53] (7B parameters, trained on UltraInteract/UltraFeedback/UltraSafety); **Skywork-Reward-Llama-3.1-8B-v0.2** [24] (8B Llama-3 model, 80K preference pairs, excelling in math/coding/safety); **Starling-RM-7B** [59] (Llama2-7B-Chat fine-tuned on Nectar dataset); **UltraRM-13B** [8] (13B Llama2-based model trained on UltraFeedback); **RLHFlowRM-8B-Deepseek** [49] (Llama-3.1-8B-Instruct on Deepseek data, math-focused); and **Math-Shepherd-7B** [44] (7B model with process supervision).

D Combining SWIFT with External Reward Models

In the Best-of-N (BoN) setting, we explore two strategies for combining the SWIFT reward (R_{SWIFT}) with an external reward signal (R_{ext}), typically from a traditional reward model. We have N reasoning paths.

- (1) **Ranking-based Combination:** Let $rank_{SWIFT}(i)$ and $rank_{ext}(i)$ be the ranks of the i -th reasoning path according to SWIFT and the external reward model, respectively (lower rank is better). The relative rank is:

$$rank_{rel}(i) = \max(rank_{SWIFT}(i), rank_{ext}(i)) \quad (6)$$

We select the path with the *lowest* relative rank:

$$i^* = \arg \min_{i \in \{1, \dots, N\}} rank_{rel}(i) \quad (7)$$

- (2) **Scaling-based Combination:** We linearly scale both rewards to the range $[0, 1]$ for the N paths¹:

$$\tilde{R}_{SWIFT}(i) = \frac{R_{SWIFT}(i) - \min_j R_{SWIFT}(j)}{\max_j R_{SWIFT}(j) - \min_j R_{SWIFT}(j)} \quad (8)$$

$$\tilde{R}_{ext}(i) = \frac{R_{ext}(i) - \min_j R_{ext}(j)}{\max_j R_{ext}(j) - \min_j R_{ext}(j)} \quad (9)$$

The aggregated reward is:

$$R_{agg}(i) = \frac{\tilde{R}_{SWIFT}(i) + \tilde{R}_{ext}(i)}{2} \quad (10)$$

We select the path with the highest combined reward:

$$i^* = \arg \max_{i \in \{1, \dots, N\}} R_{agg}(i) \quad (11)$$

These combination strategies allow us to leverage both internal (SWIFT) and external reward signals.

E Layer-wise Ablation Study

Table 8 presents a detailed breakdown of SWIFT's performance when trained using different subsets of hidden layers from the task-performing LLM. We compare three configurations: using only the **first 4 layers**, using only the **last 4 layers**, and using **all layers** (default). Results are reported for three task LLMs (Llama3-3B, Llama3-8B, Ministral-8B) under BoN@64 on MATH. The first 4 layers yield substantially lower accuracy (38.2/44.6/45.4), confirming that early-layer representations lack sufficient semantic abstraction for reasoning evaluation. In contrast, the last 4 layers achieve 55.4/60.4/61.8, similar to the full model's performance (53.6/62.6/62.8). This demonstrates that discriminative information for reasoning quality is concentrated in the final layers, allowing SWIFT to remain highly effective even with a reduced layer subset, a useful insight for further efficiency optimization.

Table 8: Layer-wise ablation on MATH (BoN@64). Accuracy (%) across three task LLMs.

Layer Subset	Llama3-3B	Llama3-8B	Ministral-8B
First 4 layers	38.2	44.6	45.4
Last 4 layers	55.4	60.4	61.8
All layers (default)	53.6	62.6	62.8

¹If all the rewards are equal, we set them all to 1.0, but this is almost impossible in practice.